Engineering Decisions and Operational Guidance

Blue Harvest · Mortgage Pricing Exceptions

Purpose: explain the implemented design, its operating limits and the checks another engineer should preserve when extending the application.

1. Architecture and Important Design Decisions

1.1 Application structure

The React frontend is served by Nginx and calls an ASP.NET Core API. Controllers handle HTTP input, identity claims and DTO mapping. Application services enforce workflow rules and use EF Core to persist records in PostgreSQL. Docker Compose runs the frontend, API and database.

PricingExceptionService owns creation, revisions, decisions and history. UserService resolves stored identities and checks roles. ApprovedDiscountService returns only the discount approved for the current revision. Services use PricingExceptionsDbContext directly; there is no separate repository or unit-of-work service.

1.2 Decisions to preserve

- Revisions protect approval integrity. A manager may change submitted terms by creating a new immutable revision. The request returns to Pending; the previous approval remains historical and cannot authorize replacement terms.

- Independent review is enforced. Only the assigned reviewer can decide, and the request creator cannot self-review. Only the original manager may revise. Known internal users may read requests and history; only MortgageProcess may retrieve an effective approved discount.

- Version and revision checks reject stale decisions and edits with 409. The request Version is also an EF concurrency token; it is not a database-generated row version.

- Idempotency protects retries. Actor, operation scope and key identify a completed operation. The same input replays the original saved result; different input with that key returns 409. Role checks still occur before replay.

1.3 Deliberate boundaries

Mortgage applications remain external GUID references. Existence and eligibility are not verified. Discount limits, one request per application and read visibility are assignment policies to confirm with the client. Approval expiry, consumption and external revocation are not implemented.

2. Diagrams and Engineering Notes

The following images reuse the latest available related diagram sources. They describe the current code; the naming refactor did not change entity relationships or database fields.

2.1 Persisted entity relationships

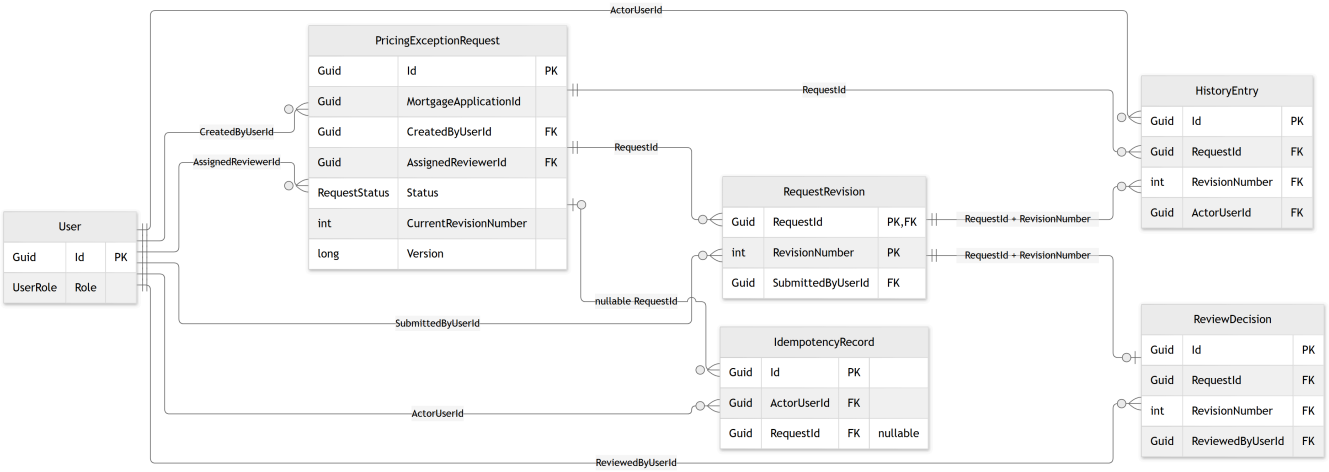


Figure 1. Current persisted entities and key relationships. Full-field detail is available in the separate entity/data-model Word document.

RequestRevision uses (RequestId, RevisionNumber) as its primary key. ReviewDecision has a unique foreign-key pair to that revision, allowing at most one decision per revision. History references both the request and revision. The current-revision pointer is maintained by services, not a composite foreign key.

2.2 Current workflow method names

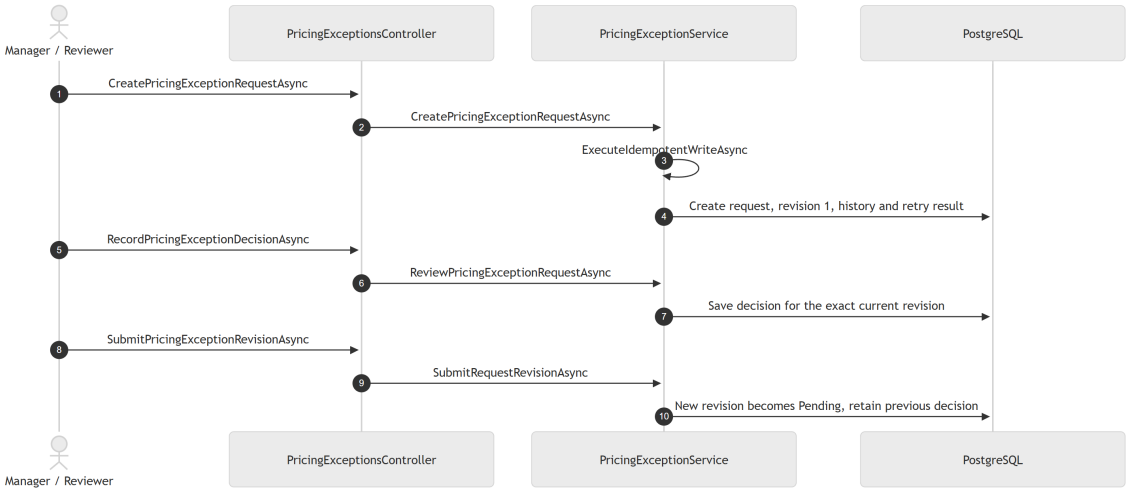


Figure 2. Creation, decision and revision calls. The manager and reviewer are separate authorized actors; the combined lane summarizes the workflow.

All mutations use ExecuteIdempotentWriteAsync. Read operations use the shared in-process lock where implemented. That lock does not provide a distributed transaction or a cross-instance snapshot.

3. Error Handling, Persistence and Operations

3.1 Errors and client recovery

DTO validation and business checks produce Problem Details responses. The global exception handler maps BusinessRuleException to 400, ForbiddenException to 403, NotFoundException to 404 and ConflictException to 409. Authentication failures produce 401. Unexpected exceptions return a generic 500 without exposing internal details; the handler logs the exception and includes a traceId in its response.

- For 400, correct the input. For 401 or 403, check the selected identity, role and assignment rather than retrying unchanged.

- For 409 after an edit or decision, reload the request and review the latest terms. Do not silently resubmit a decision against a newer revision.

- For a lost response or uncertain write outcome, retry with the original actor, key and input. Do not generate a new key merely because the first response timed out. Failed operations are not stored as completed retries.

Approved-discount lookup returns 404 when no effective current approval exists, including Pending or Declined terms. This is expected workflow behavior, not necessarily a missing mortgage application.

3.2 Persistence guarantees and limits

On PostgreSQL, one SaveChanges call saves workflow changes, history and the completed retry record transactionally. InMemory is explicitly selected by isolated tests and does not prove relational constraints or transaction behavior.

DbContext save guards reject modification or deletion of submitted revisions, decisions, history and completed retry records. This protection applies through the application; direct SQL can bypass it. One request per application and actor/scope/key retry uniqueness remain service checks without unique database indexes. The singleton lock coordinates one API process only.

3.3 Operational guidance

- Startup applies checked-in migrations automatically. Development seeding reuses stable user IDs. Plan migration compatibility and rollout before using multiple API instances.

- Keep the PostgreSQL named volume across ordinary restarts. A volume is not a backup: define backup, restore and recovery procedures before production. Deleting the volume removes business history and retry protection.

- Development accepts seeded X-User-Id identities and reads roles from storage. This is simulation, not production sign-in; demo headers are rejected outside Development and no production identity provider is configured.

- Use docker compose ps and docker compose logs --tail 50 backend postgres frontend from the repository root to inspect the deployment. A running frontend does not by itself prove that the API is ready.

- Trace and metric hooks exist for mutations, replays and conflicts. Exporters, dashboards, alerts and service-level targets are not configured. Avoid exposing request reasons or credentials in diagnostic output.

4. Safe Continuation of Development

4.1 Before changing behavior

1. Read the assignment brief, README, current service interfaces and tests. Distinguish the implemented PostgreSQL design from historical SQLite proposals in older design sections. Use the current-implementation appendix for present method names.

2. Confirm policy changes with the client: discount limits, request cardinality, read visibility, revision rights, reviewer delegation and the treatment of approvals already used downstream.

3. Preserve HTTP routes and DTO field names unless an API contract change is intended. Controller Async suffix changes can affect action-name generation; preserve the explicit GET action name used by CreatedAtAction and check Location headers.

4.2 Protect data and retry compatibility

4. Do not update old revisions or decisions to correct current terms. Create a revision and obtain a new decision. Preserve name snapshots and the ordering of audit events.

5. Treat operation scopes, serialized retry input and saved response shapes as persisted contracts. Renaming serialized payload fields or changing hashing rules can break retries stored before deployment. Payload hashing currently occurs before reason trimming; whitespace changes may therefore conflict under the same key. No retry-record expiry is implemented.

6. For schema changes, add a new EF migration and review its generated SQL and data effects. Do not rewrite an applied migration to hide a change. Back up data and test migration and recovery on a disposable database before deployment.

7. Before scaling out, design database-backed uniqueness and transaction handling for competing writers. Do not assume the in-process lock or current tests establish multi-instance safety.

4.3 Verify before handoff

8. Run dotnet build and dotnet test from Backend/BlueHarvest.PricingExceptions, with Docker available for PostgreSQL tests. From FrontEnd, run npm.cmd run build, npm.cmd run lint and npm.cmd run test:e2e after installing dependencies. Verify the Docker HTTP workflow and persistence using the repository test scripts when deployment or storage changes.

9. Check both approval and decline, revision after approval, current-discount lookup, history retention, stale changes, retry replay, conflicting retry input, role restrictions and assigned-reviewer enforcement. Investigate failures before claiming the change is validated.

10. Record the commands, environment and actual results against the handoff commit. Update diagrams and operational guidance when behavior changes. This document was checked against source files; it does not claim that tests were rerun for its creation.

4.4 Source and handoff references

Source: Controllers; Services/PricingExceptionService.cs, UserService.cs and ApprovedDiscountService.cs; Middleware/GlobalExceptionHandler.cs; Data/PricingExceptionsDbContext.cs; Observability/PricingExceptionsTelemetry.cs; Program.cs; compose.yaml. Backend paths are relative to Backend/BlueHarvest.PricingExceptions/src/PricingExceptions.API.

Related documents: Deliverables/README.md and Local_Execution_Guide_Final.docx; Documentation/BlueHarvest_Current_Entity_Relationships_Final.docx; Documentation/BlueHarvest_Current_Sequence_Diagrams_Final.docx; Documentation/Architecture/Assumptions.md. Editable images and Mermaid sources are in Backend/BlueHarvest.PricingExceptions/docs/CurrentImplementation/diagrams.